

Entity Framework Database First Approach

In this article, I am going to give you a brief introduction to the Entity Framework Database First Approach. In the Entity Framework, the Database first approach is one of the three approaches to interact with the database. The other two approaches are Model First Approach and Code First Approach. Here, in this article, we are going to keep the focus on the Database First Approach of Entity Framework and will learn how to use the DB First approach of Entity Framework to interact with the database.

What is the Database-First approach?

In Entity Framework Database First approach, the Entity Framework creates the model codes (Classes, Properties, DbContext, etc.) or you can say POCO classes from the existing database automatically. Automatically in the sense, we need to provide the necessary details about the database to the entity framework. So, in simple words, we can say that the entity framework will create the model classes based on tables and columns from the relational database.

When to use Database-First Approach of Entity Framework?

1. The Entity Framework Database First approach is used, when you already have a database; then the Entity Framework will complete its duty and create the required entities for you.
2. If you already designed the database and if you don't want to do extra efforts, then you can go with the Database First Approach of Entity Framework.
3. If you want to modify the database manually and you want to update the model from a database, then you can go with this approach.

Note: I am going to use a Console Application to demonstrate the Database First Approach of Entity Framework in this article. But it is up to you i.e. you can use any type of .NET Applications with Entity Framework like Web API, ASP.NET MVC, WCF, etc. Of Course, in our upcoming articles, we are going to use different types of .NET Applications with Entity Framework.

Example:

In this example, I am going to show you how to use Entity Framework Database First Approach using Console Application. Here, I am going to interact with SQL Server Database, but you can use any database as per your choice like MySQL, Oracle, etc,

Required Database Tables, Views, and Stored Procedures:

We are going to use the following tables in this entity framework database first approach article series. Using these tables I am going to explain each and every concept of the Entity Framework database First Approach.

Standard

StandardId	StandardName	Description
1	STD1	Outstanding
2	STD2	Good
3	STD3	Average
4	STD4	Below Average

Student

StudentId	FirstName	LastName	StandardId
1	Virat	Kohli	1
2	Rohit	Sharma	2
3	Lokesh	Rahul	3
4	Smriti	Mandana	4

Teacher

TeacherId	FirstName	LastName	StandardId
1	Anurag	Mohanty	1
2	Preeti	Tiwary	2
3	Priyanka	Dewangan	3
4	Sambit	Satapathy	3
5	Hina	Sharma	2
6	Sishanta	Jena	1

StudentAddress

StudentId	Address1	Address2	Mobile	Email
1	Lane1	Lane2	1111111111	1@dotnettutorials.net
2	Lane3	Lane4	2222222222	2@dotnettutorials.net
3	Lane5	Lane6	3333333333	3@dotnettutorials.net
4	Lane7	Lane8	4444444444	4@dotnettutorials.net

Course

CourseId	CourseName	TeacherId
1	.NET	1
2	Java	2
3	PHP	3
4	Oracle	4
5	Android	5
6	Python	6

StudentCourse

StudentId	CourseId
1	1
1	2
2	3
2	4
3	1
3	6
4	5
4	6

Please use the below SQL Script to create the necessary database, tables, and with the required dummy data.

```
-- Create the EF_Demo_DB database.
CREATE DATABASE EF_Demo_DB;
GO
-- Use EF_Demo_DB
USE EF_Demo_DB;
GO
-- Create Standard Table
CREATE TABLE Standard(
    StandardId INT PRIMARY KEY IDENTITY(1,1),
    StandardName VARCHAR(100),
    Description VARCHAR(100)
)
GO
-- Standard table data
INSERT INTO Standard VALUES('STD1', 'Outstanding');
INSERT INTO Standard VALUES('STD2', 'Good');
INSERT INTO Standard VALUES('STD3', 'Average');
INSERT INTO Standard VALUES('STD4', 'Below Average');
GO
-- Create Teacher Table
CREATE TABLE Teacher(
    TeacherId INT PRIMARY KEY IDENTITY(1,1),
    FirstName VARCHAR(100),
```

```

    LastName VARCHAR(100),
    StandardId INT FOREIGN KEY REFERENCES Standard(StandardId)
)
GO
-- Teacher table data
INSERT INTO Teacher VALUES('Anurag', 'Mohanty', 1);
INSERT INTO Teacher VALUES('Preety', 'Tiwary', 2);
INSERT INTO Teacher VALUES('Priyanka', 'Dewangan', 3);
INSERT INTO Teacher VALUES('Sambit', 'Satapathy', 3);
INSERT INTO Teacher VALUES('Hina', 'Sharma', 2);
INSERT INTO Teacher VALUES('Sishanta', 'Jena', 1);
GO
-- Create Course Table
CREATE TABLE Course(
    CourseId INT PRIMARY KEY IDENTITY(1,1),
    CourseName VARCHAR(100),
    TeacherId INT FOREIGN KEY REFERENCES Teacher(TeacherId)
)
GO
-- Course table data
INSERT INTO Course VALUES('.NET', 1);
INSERT INTO Course VALUES('Java', 2);
INSERT INTO Course VALUES('PHP', 3);
INSERT INTO Course VALUES('Oracle', 4);
INSERT INTO Course VALUES('Android', 5);
INSERT INTO Course VALUES('Python', 6);
GO
-- Create Student Table
CREATE TABLE Student(
    StudentId INT PRIMARY KEY IDENTITY(1,1),
    FirstName VARCHAR(100),
    LastName VARCHAR(100),
    StandardId INT FOREIGN KEY REFERENCES Standard(StandardId)
)
GO
-- Student table data
INSERT INTO Student VALUES('Virat', 'Kohli', 1);
INSERT INTO Student VALUES('Rohit', 'Sharma', 2);
INSERT INTO Student VALUES('Lokesh', 'Rahul', 3);
INSERT INTO Student VALUES('Smriti', 'Mandana', 4);
GO
-- Create StudentAddress Table
CREATE TABLE StudentAddress(
    StudentId INT PRIMARY KEY FOREIGN KEY REFERENCES Student(StudentId),
    Address1 VARCHAR(100),
    Address2 VARCHAR(100),
    Mobile VARCHAR(10),
    Email VARCHAR(100)
)
GO
-- StudentAddress table data

```

```

INSERT INTO StudentAddress VALUES(1, 'Lane1', 'Lane2', '1111111111',
'1@dotnettutorials.net');
INSERT INTO StudentAddress VALUES(2, 'Lane3', 'Lane4', '2222222222',
'2@dotnettutorials.net');
INSERT INTO StudentAddress VALUES(3, 'Lane5', 'Lane6', '3333333333',
'3@dotnettutorials.net');
INSERT INTO StudentAddress VALUES(4, 'Lane7', 'Lane8', '4444444444',
'4@dotnettutorials.net');
GO
-- Create StudentCourse Table
CREATE TABLE StudentCourse(
    StudentId INT NOT NULL FOREIGN KEY REFERENCES Student(StudentId),
    CourseId INT NOT NULL FOREIGN KEY REFERENCES Course(CourseId)
    PRIMARY KEY (StudentId, CourseId)
)
GO
-- StudentCourse table data
INSERT INTO StudentCourse VALUES(1,1);
INSERT INTO StudentCourse VALUES(1,2);
INSERT INTO StudentCourse VALUES(2,3);
INSERT INTO StudentCourse VALUES(2,4);
INSERT INTO StudentCourse VALUES(3,1);
INSERT INTO StudentCourse VALUES(3,6);
INSERT INTO StudentCourse VALUES(4,5);
INSERT INTO StudentCourse VALUES(4,6);
GO

```

As we are going to work with the Database First Approach i.e. working with the existing database. It might be possible that we need to use some existing stored procedures and views in our application. For this purpose, we are also going to include the following Stored Procedures and views into our database.

```

-- Get Courses by StudentId Procedure
CREATE PROCEDURE spGetCoursesByStudentId
    @StudentID INT
AS
BEGIN
    SELECT c.CourseId, c.CourseName, c.TeacherId
    FROM Student s LEFT OUTER JOIN StudentCourse sc on sc.StudentId =
s.StudentId
    LEFT OUTER JOIN Course c on c.CourseId = sc.CourseId
    WHERE s.StudentId = @StudentId
END
GO
-- Insert Student stored Procedure
CREATE PROCEDURE spInsertStudent
    @StandardId INT,
    @FirstName VARCHAR(100),
    @LastName VARCHAR(100)
AS
BEGIN
    INSERT INTO Student(FirstName ,LastName, StandardId)

```

```

        VALUES(@FirstName, @LastName, @StandardId);
        SELECT SCOPE_IDENTITY() AS StudentId
END
GO
-- Update Student stored Procedure
CREATE PROCEDURE spUpdateStudent
    @StudentId INT,
    @StandardId INT,
    @FirstName VARCHAR(100),
    @LastName VARCHAR(100)
AS
BEGIN
    UPDATE Student
    SET FirstName = @FirstName,
        LastName = @LastName,
        StandardId = @StandardId
    WHERE StudentId = @StudentId;
END
GO
-- Delete Student stored Procedure
CREATE PROCEDURE spDeleteStudent
    @StudentId int
AS
BEGIN
    DELETE FROM Student
    WHERE StudentId = @StudentId
END
GO
-- Student Course View
CREATE VIEW vwStudentCourse
AS
    SELECT s.StudentId,
        s.FirstName,
        s.LastName,
        sc.CourseId,
        c.CourseName
    FROM Student s INNER JOIN StudentCourse sc
    ON s.StudentId = sc.StudentId
    INNER JOIN Course c ON sc.CourseId = c.CourseId
GO

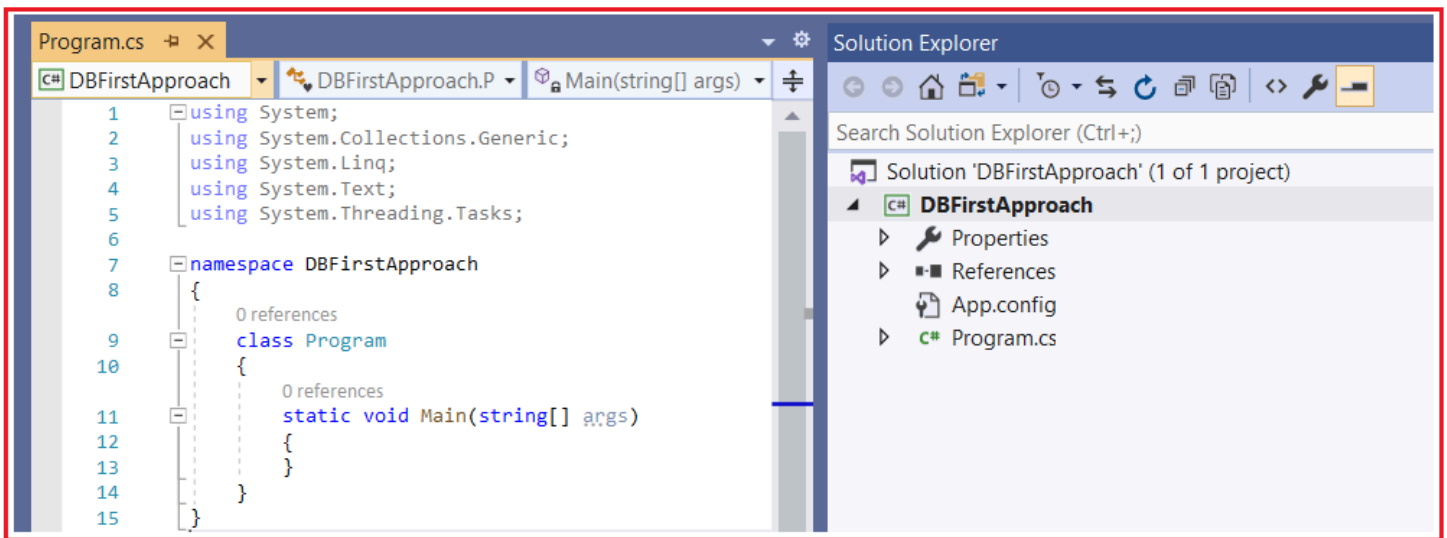
```

Creating a Console Application

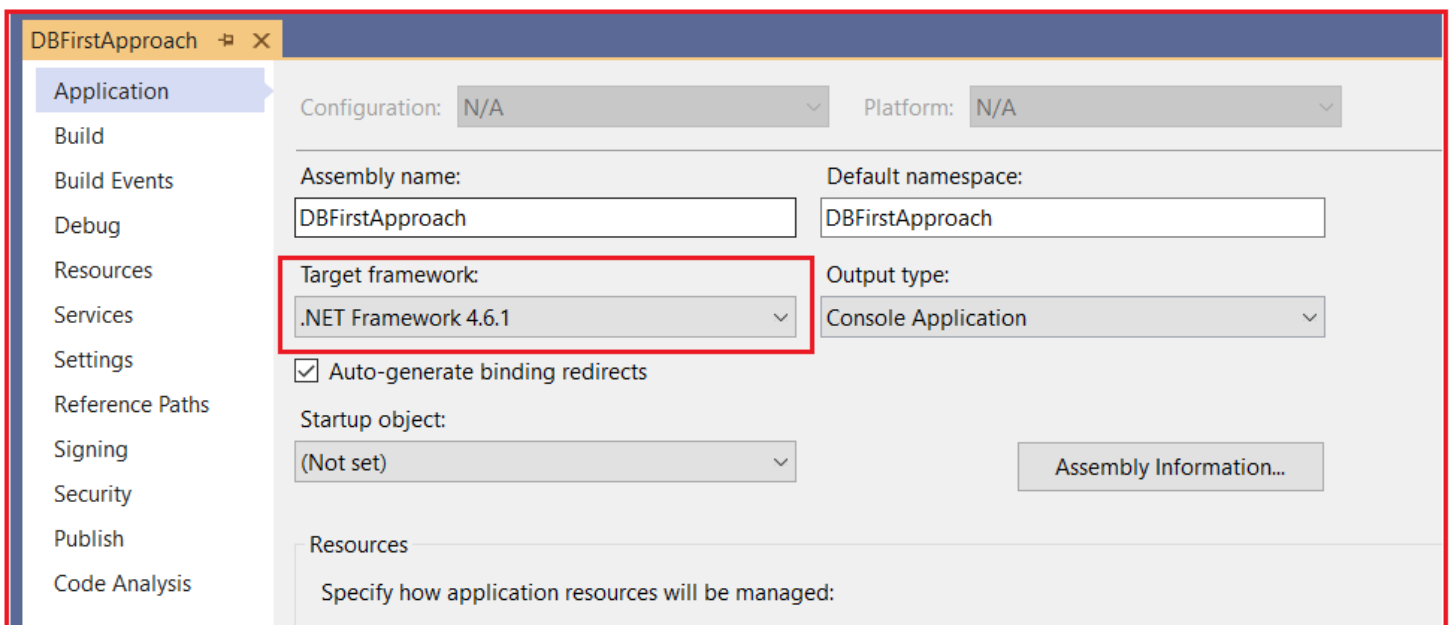
Now, we are going to create a Console Application and then we will see how to interact with the database that we just created using the Entity Framework Database First Approach.

Step1: Open Visual Studio and create a console project

First, open Visual Studio (2012\2015\2017) and then create a new console application with the name "DBFirstApproach". Once you created the project, it should look as shown below.



Once you created the console application then go to **PROJECT** menu -> **{project name} Properties** and then make sure that the project's target framework should be greater than .NET Framework 4.5 as shown below in the below image.

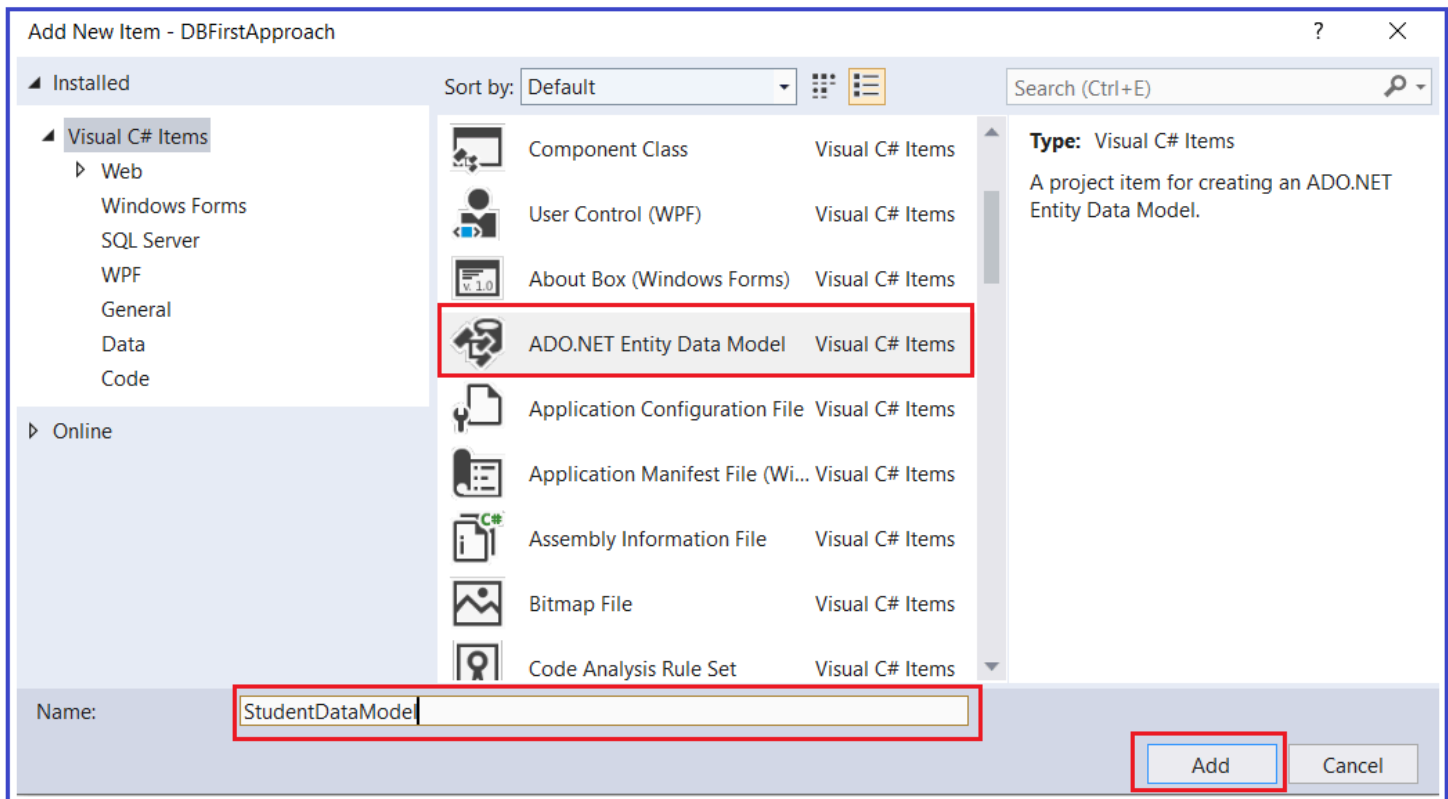


Step2: Adding Entity Data Model

The Entity Framework uses EDM to perform all the database-related operations. In simple words, we can say that the Entity Data Model is a model that describes the entities and the relationships between them.

Let us create a simple EDM (Entity Data Model) for our existing database (i.e. **EF_Demo_DB**) using the database-first approach. Here we are going to use Entity Framework 6.

In order to add the Entity Data Model, right-click on the project in the solution explorer and then select **Add -> New Item** from the context menu. This will open the **Add New Item** popup window. From this **Add New Item** window, you just need to select **ADO.NET Entity Data Model** and provide an appropriate name to the EDM (I am giving the name as **StudentDataModel**), and click on the **Add** button as shown in the below image.

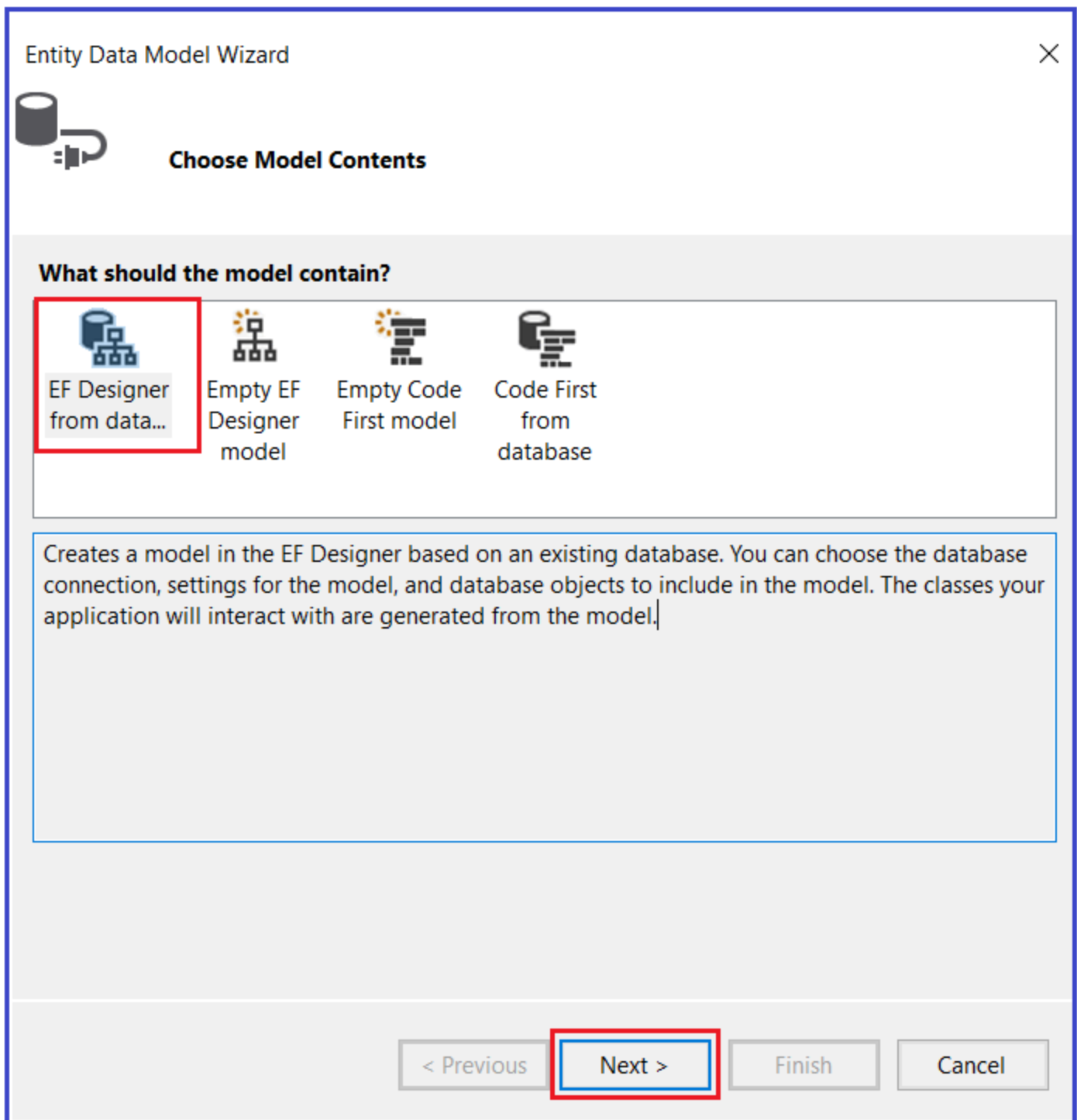


Step3: Selecting Entity Data Model Wizard

Once you click on the Add button in our previous step it will open the Entity Data Model Wizard with four options. They are as follows:

1. **EF Designer from database:** Creates a model in the EF Designer based on an existing database. You can choose the database connection, settings for the model, and database objects to include in the model. The classes your application will interact with are generated from the model.
2. **Empty EF Designer model:** Creates an empty model in the EF Designer as a starting point for visually designing your model. Later, you can generate a database from your model. The classes your application will interact with are generated from the model.
3. **Empty Code First model:** Creates an empty Code First model as a starting point for designing your model using code. Later, you can generate a database from your model.
4. **Code First from database:** Creates a Code First model based on an existing database. You can choose the database connection, settings for the model, and database objects to include in the model.

As we are going to use Entity Framework database first approach, so we need to select the **EF Designer from database** option and then click on the **Next** button as shown in the below image.



Step4: Choose your Data Connection

Once you click on the Next button in our previous step it will open the Choose your Data Connection wizard. Here, we need to create a connection with our existing database. In order to create a database connection, simply click on the **New Connection** button as shown below.

Entity Data Model Wizard



Choose Your Data Connection

Which data connection should your application use to connect to the database?

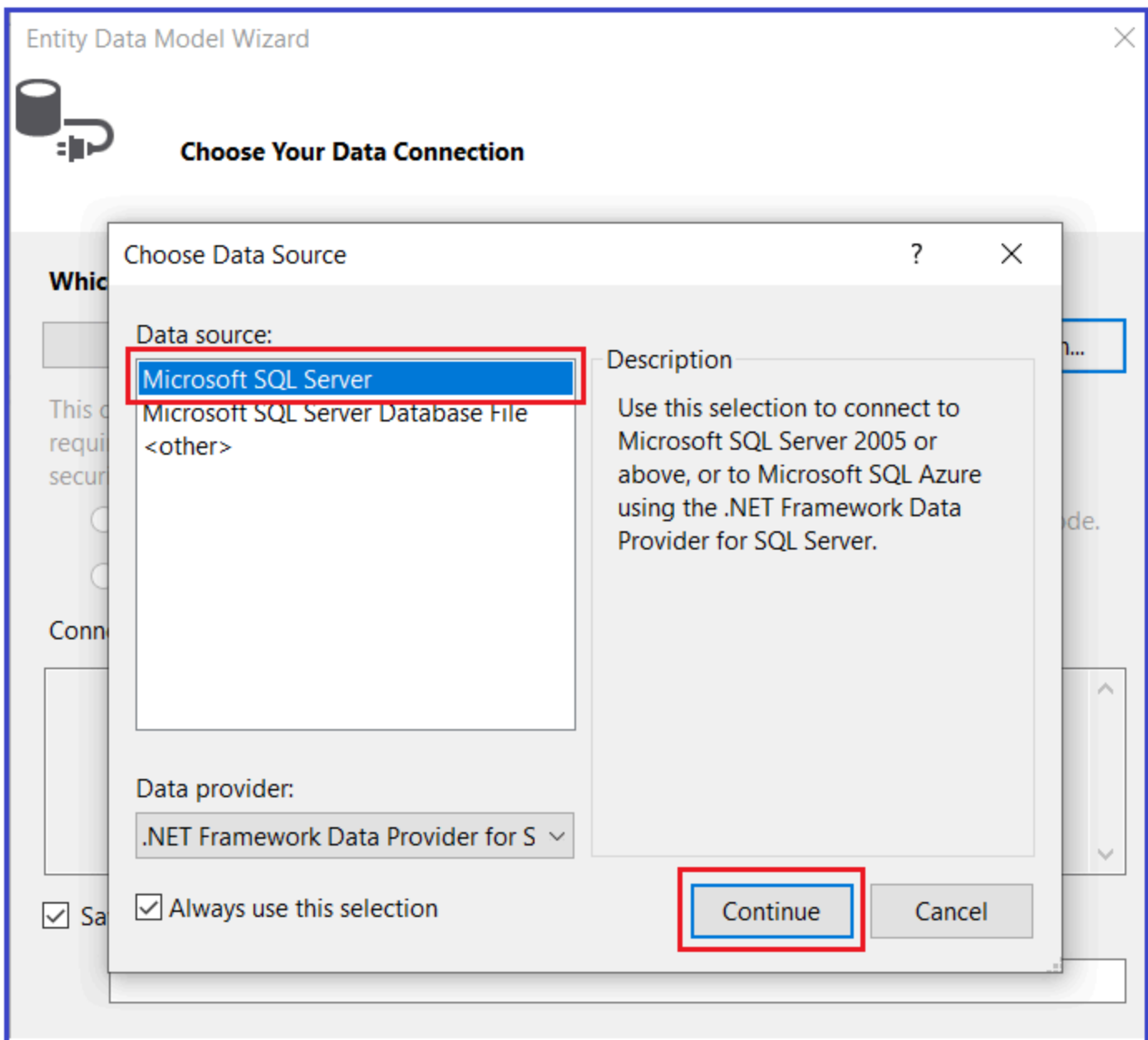
New Connection...

This connection string appears to contain sensitive data (for example, a password) that is required to connect to the database. Storing sensitive data in the connection string can be a security risk. Do you want to include this sensitive data in the connection string?

☐ No, exclude sensitive data from the connection string. I will set it in my application code.

☐ Yes, include the sensitive data in the connection string.

Once you click on the New Connection button, it will open the Choose Data Source popup. Here, you need to choose the Microsoft SQL Server data source and click on the continue button as shown in the below image.



Once you click on the Continue button, it will open the Connection Properties window. Here, you need to provide the server name. Select the Authentication type and finally select your database name and test the connection. If everything is well and fine, then you will get a message as "Test Connection Succussed". Finally, click on the **OK** button as shown below.'

Connection Properties

Enter information to connect to the selected data source or click "Change" to choose a different data source and/or provider.

Data source:
Microsoft SQL Server (SqlClient) Change...

Server name:
LAPTOP-ICA2LCQL\SQLEXPRESS Refresh

Log on to the server

Authentication: Windows Authentication

User name:

Password:

☐ Save my password

Connect to a database

☒ Select or enter a database name:
EF_Demo_DB

☐ Attach a database file:
Browse...

Logical name:

Advanced...

Test Connection OK Cancel

Once you click on the OK button, by default it will be going to add a connection string to your app.config (in case of console or windows application) or web.config (in case of any type of web application) file with the **<DB name>Entities** name. In our case, the connection string name will be EF_Demo_DBEntities. If you want then you can also change this default name, but I am going with this default name. Finally, click on the **Next** button as shown in the below image.

Entity Data Model Wizard



Choose Your Data Connection

Which data connection should your application use to connect to the database?

laptop-ica2lcql\sqlexpress.EF_Demo_DB.dbo

New Connection...

This connection string appears to contain sensitive data (for example, a password) that is required to connect to the database. Storing sensitive data in the connection string can be a security risk. Do you want to include this sensitive data in the connection string?

☐ No, exclude sensitive data from the connection string. I will set it in my application code.

☐ Yes, include the sensitive data in the connection string.

Connection string:

metadata=res://*/StudentDataModel.csdl|res://*/StudentDataModel.ssdl|res://*/StudentDataModel.msl;provider=System.Data.SqlClient;provider connection string="data source=LAPTOP-ICA2LCQL\SQLEXPRESS;initial catalog=EF_Demo_DB;integrated security=True;MultipleActiveResultSets=True;App=EntityFramework"

☒ Save connection settings in App.Config as:

EF_Demo_DBEntities

You can give any name to the Connection string

< Previous

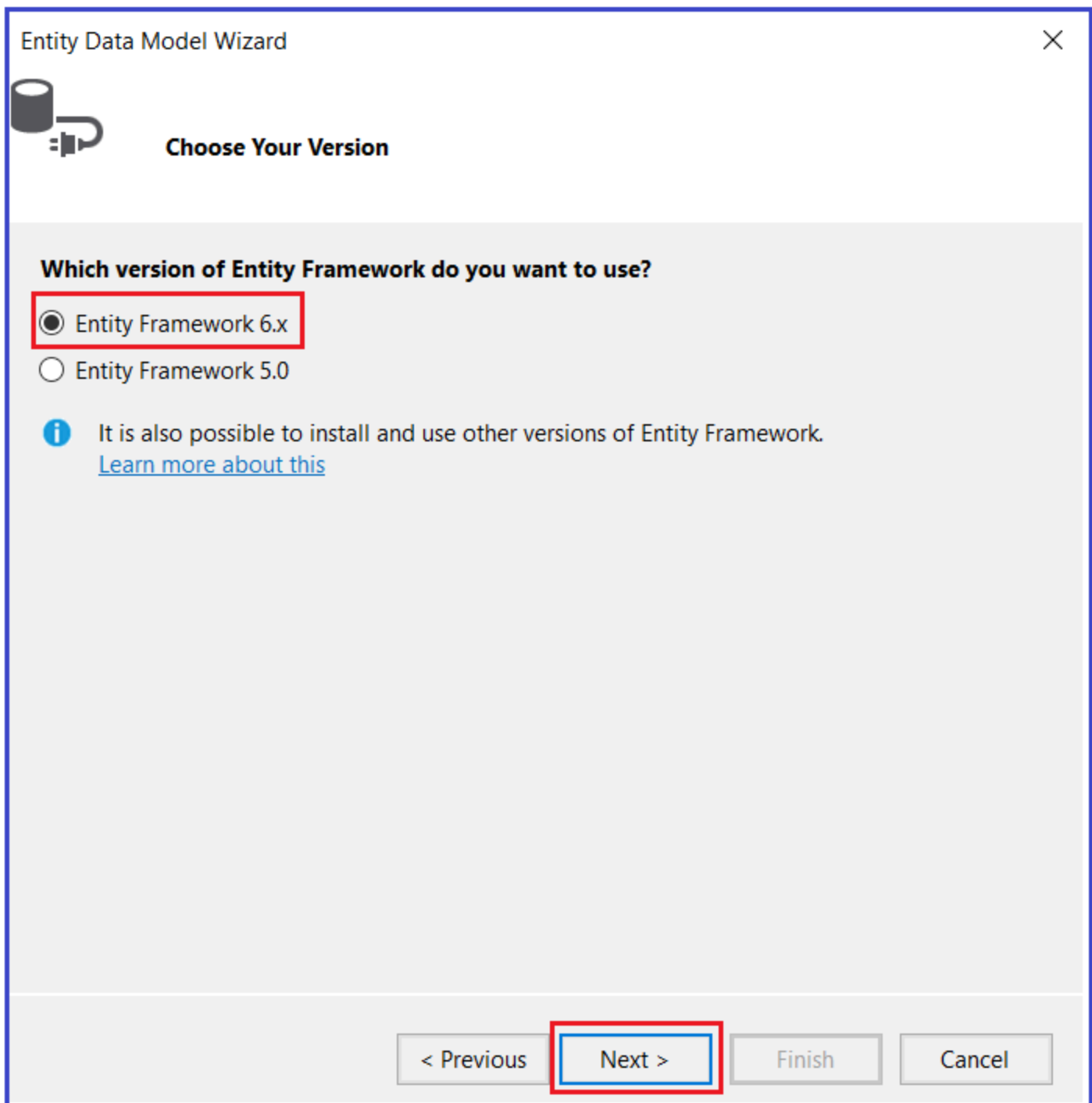
Next >

Finish

Cancel

Step5: Choose Entity Framework Version

Once you click on the Next button in our previous step, it will open choose your entity framework version wizard. Here, you need to select the **Entity Framework 6.x** and click on the **Next** button as shown below.



Note: If you already have installed the latest version of Entity Framework using NuGet manager, then this step of the wizard will no longer appear.

Step6: Choose your Database Objects and Settings

Once you click on the Next button in our previous step, it will open Choose your Database Objects and Settings wizard. This wizard will display all the Tables, Views, and Stored Procedures (SP) that are available in the database. So, select the Tables, Views, and SPs you want, and keep the default checkboxes selected and finally click on the **Finish** button as shown in the below image. You can also change the Model Namespace if you want as per your choice. I am going with the default model namespace

Entity Data Model Wizard

Choose Your Database Objects and Settings

Which database objects do you want to include in your model?

- ☒ Tables
 - ☒ dbo
 - ☒ Course
 - ☒ Standard
 - ☒ Student
 - ☒ StudentAddress
 - ☒ StudentCourse
 - ☒ Teacher
 - ☒ Views
 - ☒ dbo
 - ☒ vwStudentCourse
 - ☒ Stored Procedures and Functions
 - ☒ dbo
 - ☒ spDeleteStudent

Select all check boxes from Tables, Views and Stored Procedure and Functions to include all the tables, views and stored procedure and functions that we have in our existing database

- ☒ Pluralize or singularize generated object names
- ☒ Include foreign key columns in the model
- ☒ Import selected stored procedures and functions into the entity model

Check all the Check boxes here also

Model Namespace:

EF_Demo_DBModel

If you want then you can check the Model Namesoace

< Previous Next > **Finish** Cancel

Points to Remember:

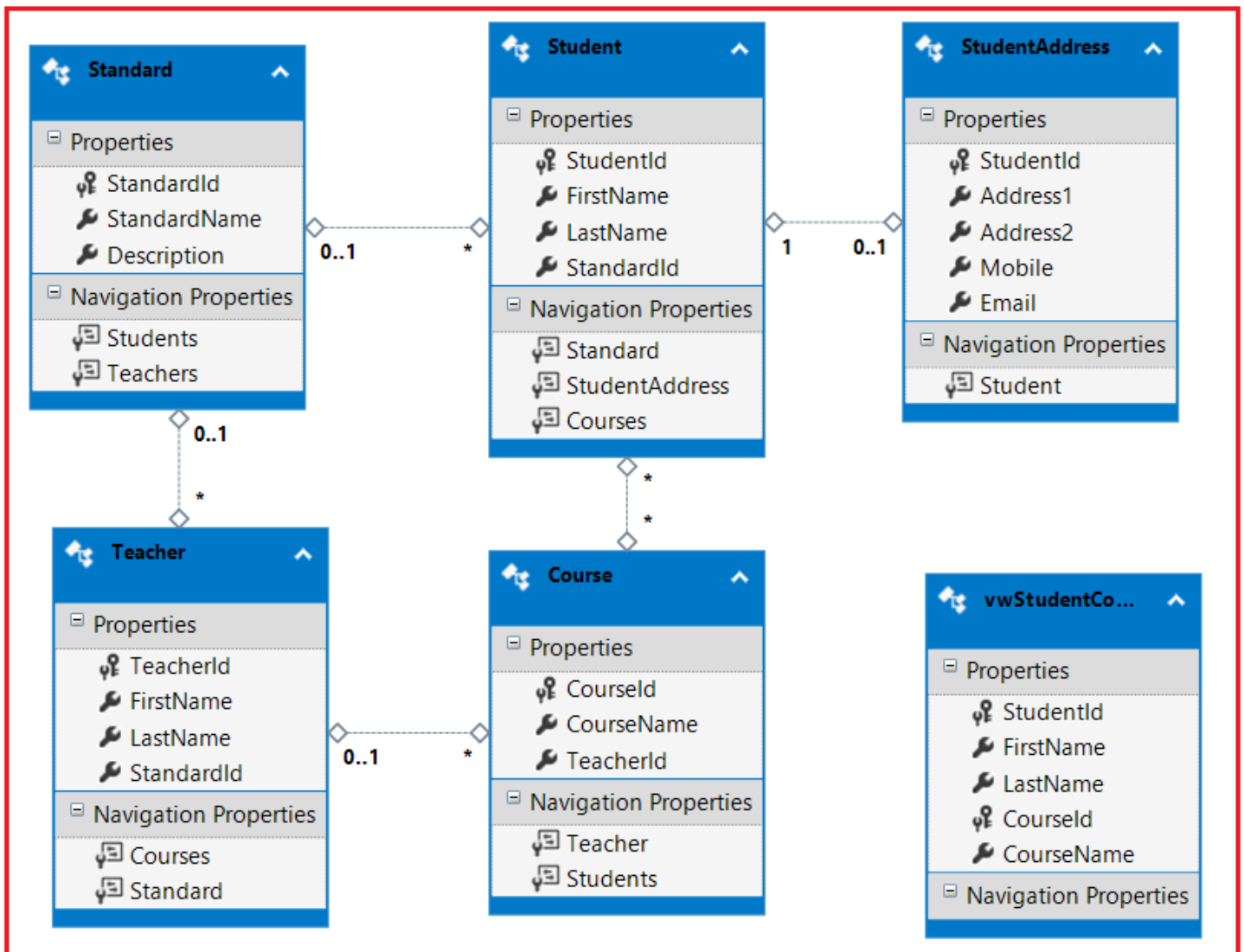
Pluralize or singularize generated object names checkbox singularizes an entity set name if the table name in the database is plural. For example, if the EF_Demo_DB database contains the Students table name, then the entity set would be a singular Student. Similarly, relationships between the models will be pluralized if the table has a one-to-many or a many-to-many relationship with other tables. For example, the Student table has a many-to-many relationship with the Course table, so the Student entity set will have the plural name Courses for the collection navigation property of Course type.

The second checkbox, **Include foreign key columns in the model**, includes a foreign key property explicitly to represent the foreign key. For example, the Student table has a one-to-many relationship with the Standard table. To represent this in the model, the Student entity set includes a StandardId property with Standard reference navigation property. If this checkbox is unchecked, then it will only include the Student reference navigation property, but not the StandardId.

The third checkbox, **Import selected stored procedures and functions into entity model**, automatically creates Function Imports for the stored procedures and functions. You don't need to import functions manually like it was necessary prior to Entity Framework 6.0.

Step7: EDMX file is generated

Once you click on the **Finish** button, a **StudentDataModel.edmx** file will be added to your project. Open EDM designer by double-clicking on **StudentDataModel.edmx**. This displays all the entities for the selected tables and the relationships between them as shown below. In a later article, I will explain the relationship in detail.



App.config file:

The EDM adds the provider and connection string settings in the App.config file as shown below:

```
<?xml version="1.0" encoding="utf-8"?>
<configuration>
```

```

<configSections>
  <!-- For more information on Entity Framework configuration, visit http://
  <section name="entityFramework" type="System.Data.Entity.Internal.ConfigFi
Culture=neutral, PublicKeyToken=b77a5c561934e089" requirePermission="false" />
</configSections>
<startup>
  <supportedRuntime version="v4.0" sku=".NETFramework,Version=v4.6.1" />
</startup>
<connectionStrings>
  <add name="EF_Demo_DBEntities"
connectionString="metadata=res://*/StudentDataModel.csdl|res://*/StudentDataModel.ssd
connection string="data source=LAPTOP-ICA2LCQL\SQLEXPRESS;initial catalog=
security=True;MultipleActiveResultSets=True;App=EntityFramework" provider=
</connectionStrings>
<entityFramework>
  <defaultConnectionFactory type="System.Data.Entity.Infrastructure.SqlConnec
<providers>
  <provider invariantName="System.Data.SqlClient" type="System.Data.Entity
</providers>
</entityFramework>
</configuration>

```

That's it, you have successfully created an Entity Data Model file (.edmx) for your existing database.

Using Entity Framework in Code:

Let us see a simple example of how to use the entity framework database first approach. Let us modify the main method of the Program class as shown below. Here, we need to display all the student details in the console. I am not going to explain the below code at the moment. Later In our upcoming articles, I will explain the code in detail for sure.

```

using System;
using System.Collections.Generic;
using System.Linq;
namespace DBFirstApproach
{
    class Program
    {
        static void Main(string[] args)
        {
            using (EF_Demo_DBEntities DBEntities = new EF_Demo_DBEntities())
            {
                List<Student> listStudents = DBEntities.Students.ToList();
                Console.WriteLine();
                foreach (Student student in listStudents)
                {
                    Console.WriteLine($" Name = {student.FirstName}
{student.LastName}, Email {student.StudentAddress?.Email}, Mobile
{student.StudentAddress?.Mobile}");
                }
                Console.ReadKey();
            }
        }
    }
}

```

```
}  
}  
}
```

Output:

```
Name = Virat Kohli, Email 1@dotnettutorials.net, Mobile 1111111111  
Name = Rohit Sharma, Email 2@dotnettutorials.net, Mobile 2222222222  
Name = Lokesh Rahul, Email 3@dotnettutorials.net, Mobile 3333333333  
Name = Smriti Mandana, Email 4@dotnettutorials.net, Mobile 4444444444
```

What is Entity Data Model?

The Entity Data Model is a model that describes the entities and the relationships between them and the Entity Framework uses this EDM (Entity Data Model) to perform all the database-related operations

In the next article, I am going to discuss the [Entity Data Model of Entity Framework](#) in detail. In this article, I try to explain how to interact with the SQL Server database using the Entity Framework Database First Approach and I hope you enjoyed this Introduction Entity Framework Database First Approach article. Please give your valuable feedback and suggestions about this article.